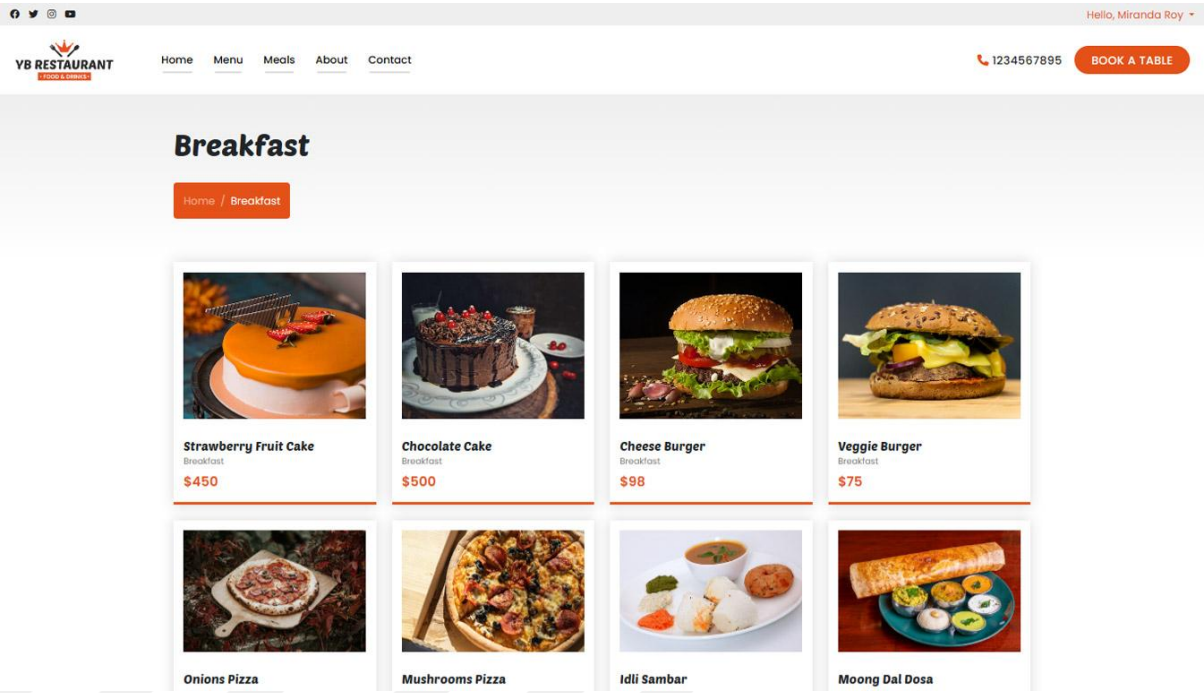
Grade 3

IT Department
Web Development

On-site 3

ON-SITE GUIDE
2025 - 2026

Web Development On-Site 3
Laravel Restaurant Management System



Your Mission:

Your first project is to build a complete Laravel web application with two main parts:

1. **A public-facing website where customers can view all the meals.**
2. **A secure admin dashboard where restaurant staff can log in and manage meal categories and the meals themselves.**

Table of Contents

Section 1: Installing Laravel and Project Setup

- Installing Laravel Installer
- Creating a New Laravel Project
- Configuring the Environment
- Installing and Testing Laravel Breeze Authentication

Section 2: Setting Up Authentication with Breeze

- Installing Breeze
- Testing Register/Login
- Creating Home Page for Public View

Section 3: Database Design and Migrations

- Designing Category and Meal Tables
- Creating Migrations with Models & Controllers
- Running Migrations and Viewing Tables

Section 4: Building Admin Dashboard Layout

- Creating a Layout File with Bootstrap
- Adding Navigation, Dashboard, and Blade Structure

Section 5: Category Management

- Implementing CategoryController (CRUD)
- Building Blade Views for Category Management
- Slug Generation & Validation

Section 6: Meal Management

- MealController with Image Upload
- Create/Edit/Delete Meals
- Meal-Category Relationship
- Validations and File Handling

Section 7: Display Meals on Home Page

- Showing Meals with Categories
- Displaying Images
- Using Bootstrap Cards

Section 8: challenges for students

Section 1: Installing Laravel and Project Setup

By the end of this section, students will be able to:

- Install Laravel globally
- Creating a new Laravel project
- Configure environment variables
- Set up database connection
- Start the local development server

Tip: Close Antivirus and Run Xampp

1.1 Install Laravel

Open your terminal then run:

```
composer global require laravel/installer
```

1.2 Create a New Laravel Project

Go to the location where you want to install the project.

Open your terminal then run:

```
laravel new restaurant
```

During the installation, choose:

- **Starter kit**: None
- **Testing framework**: Pest
- **Database**: MySQL
- **Run default migrations**: No
- **Run npm install and build**: Yes

1.3 Navigate and Start Laravel Server

Open your terminal then run:

```
cd restaurant
composer run dev
```

Open in browser:

```
http://localhost:8000
```

1.4 Configure .env File

Open .env in your project folder and update:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=restaurant
DB_USERNAME=root
DB_PASSWORD=
```

Tip: Make sure your MySQL service is running, and the restaurant database exists.

Section 2: Installing Authentication with Laravel Breeze

By the end of this section, students will be able to:

- Understand Laravel Breeze and why it's useful
- Install and configure Breeze with Blade views
- Test user registration, login, and logout
- Prepare for protected admin routes

2.1 What is Laravel Breeze?

Laravel Breeze is a minimal, simple starter kit that provides:

- User registration
- Login/Logout
- Blade templates and Tailwind (we'll use **Blade only**)

It's perfect for anyone who wants **quick authentication** without a heavy frontend framework.

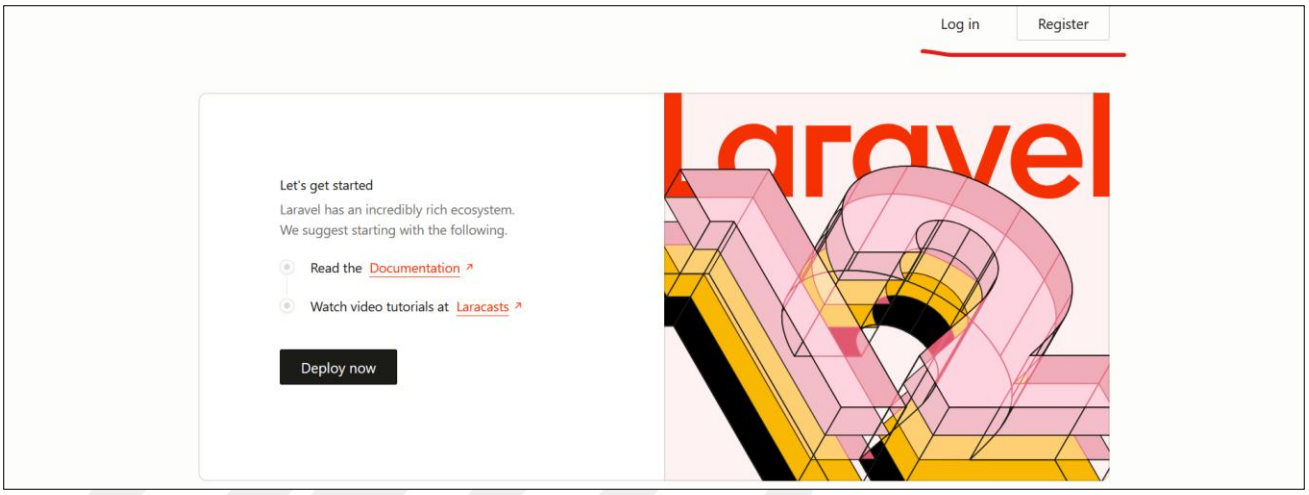
2.2 Install Breeze

Run the following in the terminal (from your project root):
Open your terminal then run:

```
composer require laravel/breeze --dev
php artisan breeze:install blade
npm install
php artisan migrate
```

```
composer run dev (stop and run again)
```

Tip: If npm install fails due to **antivirus/firewall**, disable it temporarily.



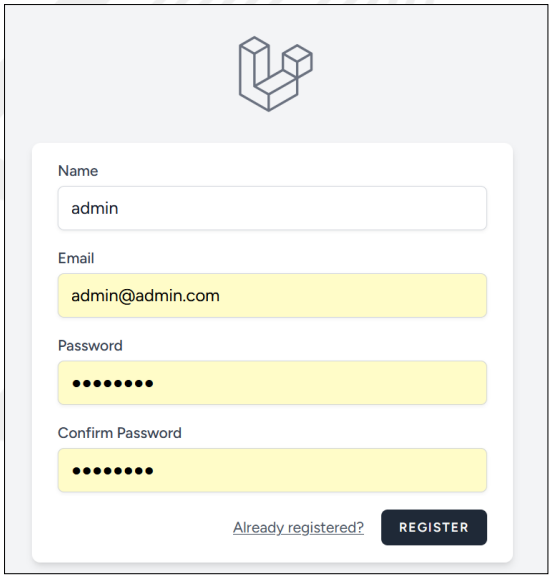
2.3 Test Authentication Pages

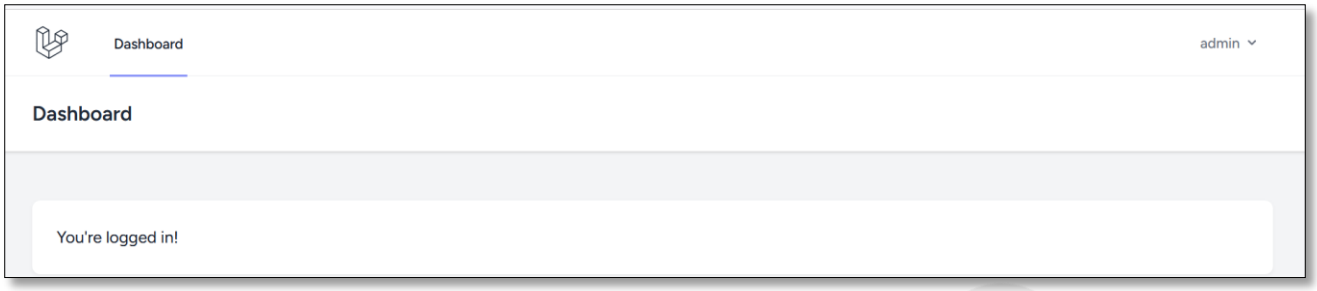
Now open your browser and test:
Register page: http://localhost:8000/register
Login page: http://localhost:8000/login
Use this test user to register:

Field	Value
Name	admin
Email	admin@laravel.com
Password	12345678

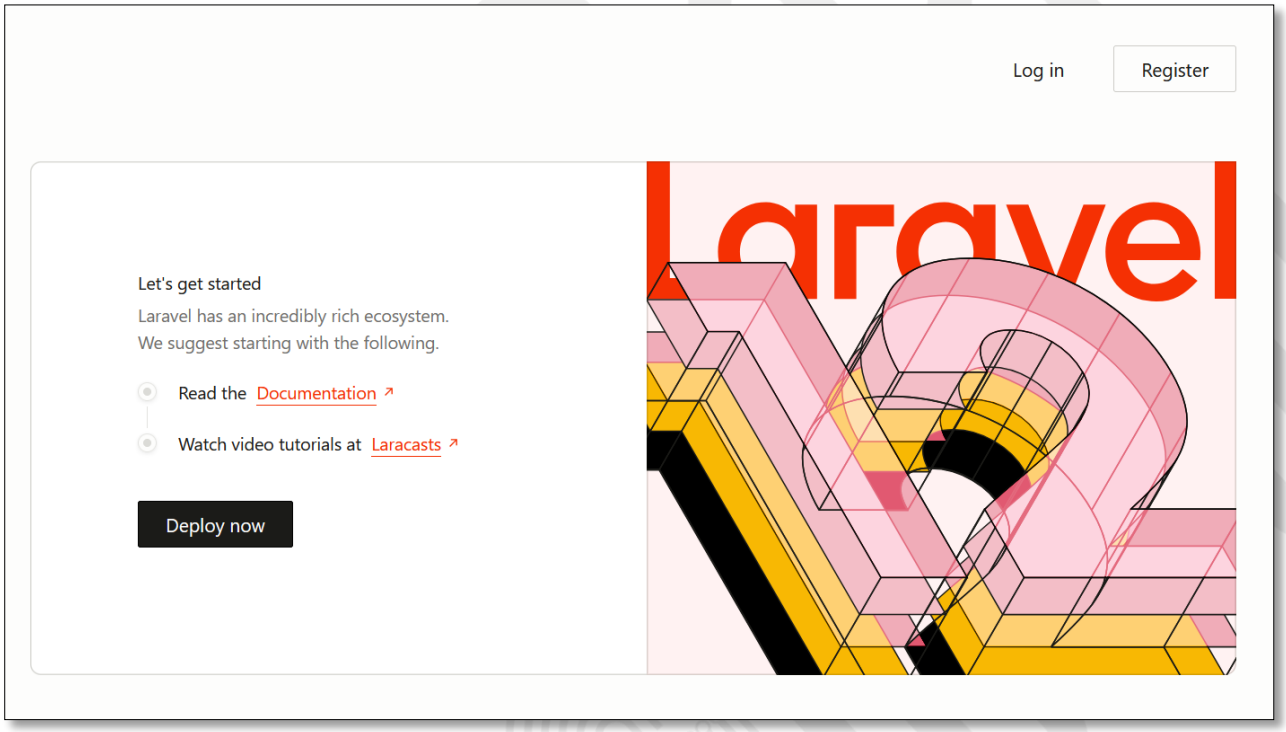
You can now login and logout as the admin.

Test to register user :admin admin@laravel.com 12345678





Test to logout and login



2.4 Enable Protected Routes

Later in the project, we will group routes for managing categories and meals under this:

```
Route::middleware('auth')->group(function () {
    // Admin dashboard, categories, meals
});
```

This ensures only logged-in users can access the admin system.

2.5 Create the Home Page Controller

To set up a public home page:

In terminal:

```
php artisan make:controller HomeController
```

In HomeController.php:

```
public function index() {
    return view('home');
}
```

Create view:

In resources/views/, add home.blade.php with simple content:

```
<h1>Home Page</h1>
```

In routes/web.php:

```
use App\Http\Controllers\HomeController;
Route::get('/', [HomeController::class, 'index'])->name('home');
```

Section 3: Database Design & Migrations

By the end of this section, students will be able to:

- Understand how Laravel migrations work
- Create tables for categories and meals
- Define relationships between tables
- Seed example data (optional)

3.1 Planning the Database

For this project, we'll need the following tables:

Categories Table

- id: primary key
- name: category name such as (Pizza, Drinks)
- slug: URL-friendly version of the name
- created_at / updated_at

Meals Table

- id: primary key
- category_id: foreign key
- name: meal name
- description: short text
- price: decimal
- image: file path
- created_at / updated_at

3.2 Create Models with Migrations

In terminal Generate Category model:

```
php artisan make:model Category -mcr
```

In terminal Generate Meal model:

```
php artisan make:model Meal -mcr
```

Tip: The -mcr flag generates:

-m: migration file
-c: controller
-r: resource controller (includes CRUD methods)

3.3 Edit Migration Files

database/migrations/xxxx_create_categories_table.php:

```
Schema::create('categories', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('slug')->unique();
    $table->timestamps();
});
```

database/migrations/xxxx_create_meals_table.php:

```
Schema::create('meals', function (Blueprint $table) {
    $table->id();
    $table->foreignId('category_id')->constrained()-
>onDelete('cascade');
    $table->string('name');
    $table->text('description');
    $table->double('price');
    $table->string('image')->nullable();
    $table->timestamps();
});
```

3.4 Run Migrations

In the terminal:

```
php artisan migrate
```

You should now have categories and meals tables in your database.

3.5 Define Relationships in Models

In Category.php:

class Category extends Model{

```
    protected $fillable = ['name', 'slug', 'description'];
```

```
    public function meals(){
        return $this->hasMany(Meal::class);
    }
```

} // class Category

In Meal.php:

class Meal extends Model{

```
    protected $fillable = [
        'category_id', 'name', 'slug', 'description', 'price', 'image'
    ];
```

```
    public function category(){
        return $this->belongsTo(Category::class);
    }
```

}//class Meal

3.6 (Optional) Create Example Categories

You can add test data manually using Tinker:

In the terminal:

```
php artisan tinker
```

```
\App\Models\Category::create([
    'name' => 'Pizza',
    'slug' => 'pizza',
]);
```

Repeat with other categories like Drinks, Burgers, Desserts, etc.

Section 4: Building the Admin Dashboard Layout

By the end of this section, students will be able to:

- Create a shared admin layout using Blade
- Build a Bootstrap navigation sidebar
- Add Logout functionality
- Create a welcome dashboard page

4.1 Create admin-layout.blade.php in layouts folder

In resources/views/**layouts**/, create a file named: **admin-layout.blade.php**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>@yield('title', 'Admin Dashboard')</title>
   <!-- Bootstrap 5 CSS CDN -->
   <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
  <div class="container">
    <div class="row">
      <!-- Sidebar -->
      <nav class="col-md-3 d-md-block bg-light sidebar py-4">
        <h4 class="text-center">Restaurant Admin</h4>
        <ul class="nav flex-column px-3">
          <li class="nav-item"><a href="{{ route('dashboard') }}"
class="nav-link">Dashboard</a></li>
          <li class="nav-item"><a href="{{ route('categories.index') }}"
class="nav-link">Categories</a></li>
          <li class="nav-item"><a href="{{ route('meals.index') }}"
class="nav-link">Meals</a></li>
        <li class="nav-item">
        <form method="POST" action="{{ route('logout') }}">
            @csrf
         <button class="btn btn-danger mt-3">Logout</button>
         </form>
        </li>
        </ul>
      </nav>
```

```
    <!-- Content -->
    <main class="col-md-9 py-4">
      @yield('content')
    </main>
  </div>     <!-- row -->
</div>     <!-- container -->
```

```
  <!-- Bootstrap JS  -->
  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.
js"></script>
```

```
</body>
</html>
```

4.2 Create Admin Dashboard Page

Create View File: In resources/views/dashboard.blade.php

```
@extends('layouts.admin-layout')
@section('title', 'Dashboard')
@section('content')
  <h2>Welcome to the Admin Dashboard</h2>
  <p>Select a section from the sidebar to manage categories or
meals.</p>
@endsection
```

Tip: An error will appear in the **routes**, and we will solve the error by adding the following code in **web.php**

4.3 Protect All Admin Pages (in routes/web.php):

```
use App\Http\Controllers\HomeController;
use App\Http\Controllers\CategoryController;
use App\Http\Controllers\MealController;
```

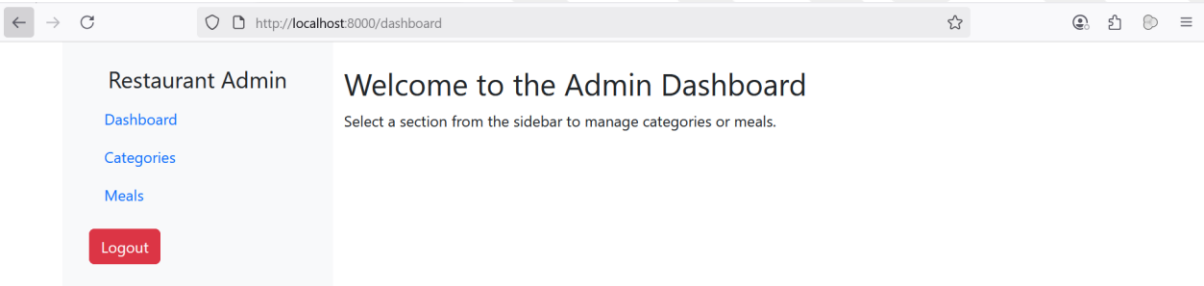
```
Route::get('/dashboard', function () {
    return view('dashboard');
})->middleware(['auth', 'verified'])->name('dashboard');
```

Make sure all admin routes are grouped inside the auth middleware:

```
Route::middleware('auth')->group(function () {
    ……————————————————————————————
// Resource routes
    Route::resource('categories', CategoryController::class);
    Route::resource('meals', MealController::class);
……————————————————————————————
});
```

require __DIR__.'/auth.php';

Test to access to dashboard : http://localhost:8000/dashboard



After the project is completed, the page will be redesigned with quick links to the main parts of the project and an image that expresses the project's activity, such as adding a category or meal.

Section 5: Category Management (CRUD)

By the end of this section, students will be able to:

- Display a list of categories
- Create a new category
- Edit and delete categories
- Use the admin layout for category pages

5.1 Resource Routes for Categories

Make sure in routes/web.php, inside the auth group:

```
Route::resource('/categories', CategoryController::class);
```

To verify or know the routes, use the following command:

In the terminal:

```
php artisan route:list
```

This auto-generates these routes:

HTTP Verb	URI	Method
GET	/categories	index
GET	/categories/create	create
POST	/categories	store
GET	/categories/{id}	show *(not used)*
GET	/categories/{id}/edit	edit
PUT/PATCH	/categories/{id}	update
DELETE	/categories/{id}	destroy

5.2 Create Views for Categories (create categories folder)

In resources/views/**categories**/, create these Blade files:

```
index.blade.php – list all categories
create.blade.php – form to add
edit.blade.php – form to update
```

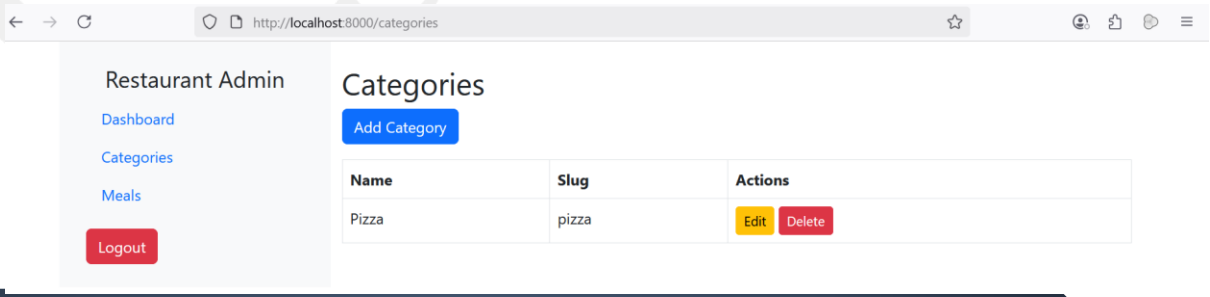
5.3 get and list all categories

In CategoryController Add index method to get categories

```
use App\Models\Category;
```

```
public function index() {
    $categories = Category::latest()->get();
    return view('categories.index', compact('categories'));
}
```

In resources\views\categories\index.blade.php



```
@extends('layouts.admin-layout')
@section('title', 'Categories')
@section('content')
<h2>Categories</h2>
<a href="{{route('categories.create')}}" class="btn btn-primary mb-3"
>Add Category</a>
<table class="table table-bordered">
  <thead>
    <tr>
      <th>Name</th>
      <th>Slug</th>
      <th>Actions</th>
    </tr>
  </thead>

  <tbody>
    @foreach ($categories as $category)
    <tr>
      <td>{{ $category->name }}</td>
      <td>{{ $category->slug }}</td>

<td>
        <a href="{{ route('categories.edit', $category) }}" class="btn
btn-warning btn-sm">Edit</a>

        <form action="{{ route('categories.destroy', $category) }}"
method="POST" class="d-inline">
          @csrf @method('DELETE')

<button class="btn btn-danger btn-sm" onclick="return
confirm('Delete?')">Delete</button>
        </form>
      </td>
    </tr>
    @endforeach
  </tbody>
</table>
@endsection
```

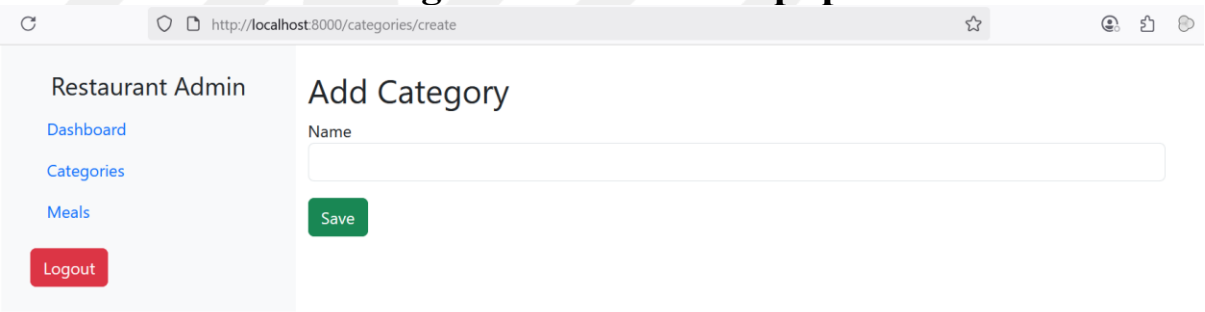
5.4 Add new category CategoryController Methods

In CategoryController Add create methods to display create form

```
use Illuminate\Support\Str;
```

```
public function create() {
   return view('categories.create');
}
```

In resources\views\categories\create.blade.php




```
@extends('layouts.admin-layout')
@section('title', 'Create Category')
@section('content')
<h2>Add Category</h2>
```

```
<form method="POST" action="{{route('categories.store') }}">
  @csrf
```

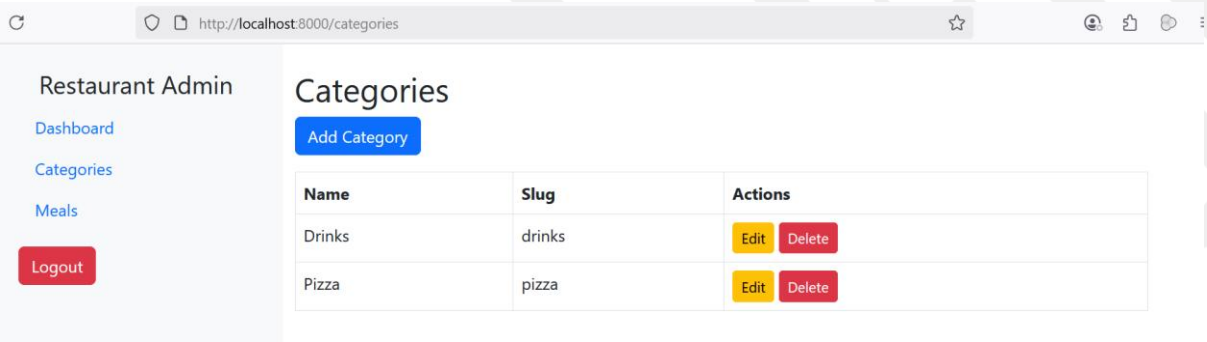
```
  <div class="mb-3">
 @error('name')
   <div class="alert alert-danger m-3">{{ $message }}</div>
@enderror
 <label>Name</label>
<input type="text" name="name"
class="form-control @error('title') is-invalid @enderror" >
  </div>
```

```
<button class="btn btn-success" type="submit"> Save </button>
</form>
```

```
@endsection
```

In CategoryController Add store method to send data in database

```
public function store(Request $request) {
    $request->validate(['name' => 'required|unique:categories']);
    Category::create([
        'name' => $request->name,
        'slug' => Str::slug($request->name),
    ]);
    session()->flash('success', 'Category created successfully.');
    return redirect()->route('categories.index');
}
```



In index.blade.php display flash message

Add code above table:

```
@if(session('success'))
    <div class="alert alert-success">
        {{ session('success') }}
    </div>
@endif
```

```
<table class="table table-bordered">
```

5.5 Update Category

In CategoryController Add edit method to display edit form

```
public function edit(Category $category) {
    return view('categories.edit', compact('category'));
}
```

In resources\views\categories\edit.blade.php

```
@extends('layouts.admin-layout')
@section('title', 'Edit Category')
@section('content')
<h2>Edit Category</h2>
```

```
<form method="POST" action="{{route('categories.update',
$category)}}">
  @csrf
  @method('PUT')
```

```
<div class="mb-3">
```

```
      @error('name')
    <div class="alert alert-danger m-3">{{ $message }}</div>
@enderror
```

```
<label>Name</label>
```

```
    <input type="text"  name="name"
value="{{ old('name', $category->name ?? '') }}"
class="form-control @error('title') is-invalid @enderror"  >
```

```
  </div>
  <button type="submit" class="btn btn-success">Update</button>
</form>
@endsection
```

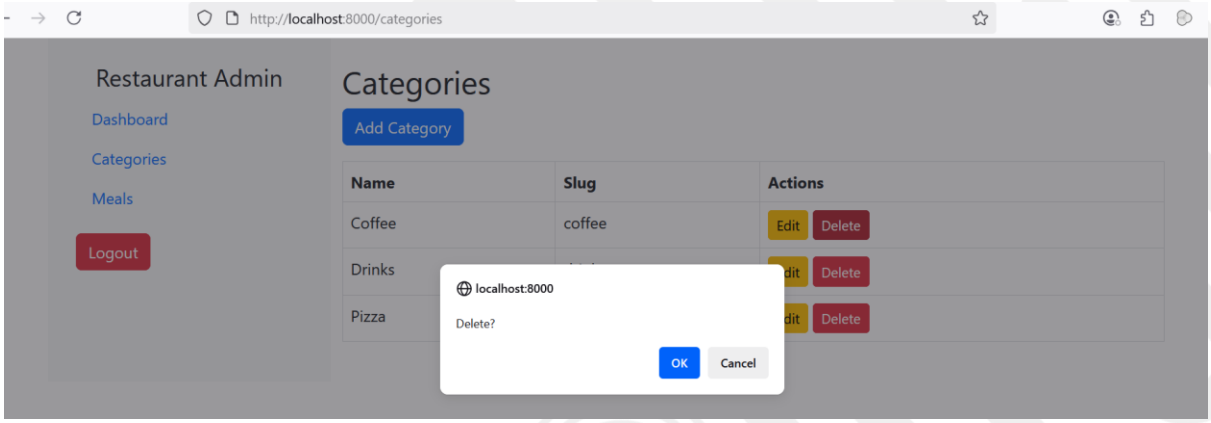
In CategoryController in Upade method

```
public function update(Request $request, Category $category) {
    $request->validate([
'name' => 'required|unique:categories,name,' . $category->id
]);
$category->update([
        'name' => $request->name,
        'slug' => Str::slug($request->name),
```

```
    ]);
    session()->flash('success', 'Category Updated successfully.');
    return redirect()->route('categories.index');
}
```

In CategoryController in destroy method

```
public function destroy(Category $category) {
    $category->delete();
    session()->flash('success', 'Category Deleted successfully.');
    return redirect()->route('categories.index');
}
```



Section 6: Meal Management (CRUD with Image Upload)

By the end of this section, students will be able to:

- Build a full CRUD for meals
- Upload and display meal images
- Connect meals to categories

6.1 Resource Routes for Meals

Make sure in routes/web.php, inside the auth group:

```
Route::resource('/meals', MealController::class);
```

To verify or know the routes, use the following command:
In the terminal:

```
php artisan route:list
```

6.2 Views for Meals (create meals folder)

Folder: resources/views/meals/

Create files:

```
index.blade.php
create.blade.php
edit.blade.php
```

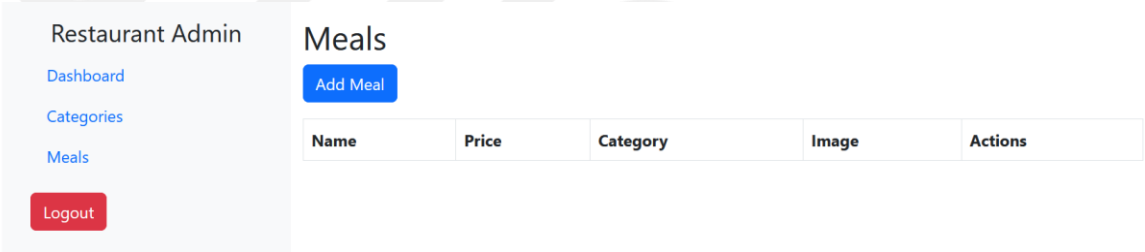
6.3 get and list all meals

In **MealController** Add index method to get meals

```
use App\Models\Meal;
use App\Models\Category;
```

```
public function index() {
    $meals = Meal::with('category')->latest()->get();
    return view('meals.index', compact('meals'));
}
```

In resources\views\meals\index.blade.php




```
@extends('layouts.admin-layout')
@section('title', 'Meals')
@section('content')
<h2>Meals</h2>
```

```
<a href="{{route('meals.create')}}" class="btn btn-primary mb-3">
Add Meal</a>
```

```
@if(session('success'))
    <div class="alert alert-success">
        {{ session('success') }}
```

```
    </div>
@endif
```

```
<table class="table table-bordered">
  <thead>
    <tr>
      <th>Name</th><th>Price</th><th>Category</th><th>Image</th
><th>Actions</th>
    </tr>
  </thead>
  <tbody>
    @foreach ($meals as $meal)
    <tr>
```

```
      <td>{{ $meal->name }}</td>
      <td>${{ $meal->price }}</td>
      <td>{{ $meal->category->name }}</td>
```

```
      <td>
        @if($meal->image)
          <img src="{{ asset('storage/' . $meal->image) }}" width="80">
        @endif
      </td>
```

```
<td>
        <a href="{{ route('meals.edit', $meal) }}" class="btn btn-
warning btn-sm">Edit</a>

        <form action="{{ route('meals.destroy', $meal) }}"
method="POST" class="d-inline">
          @csrf @method('DELETE')

          <button class="btn btn-danger btn-sm" onclick="return
confirm('Delete?')">Delete</button>
        </form>
      </td>
```

```
    </tr>
    @endforeach
  </tbody>
```

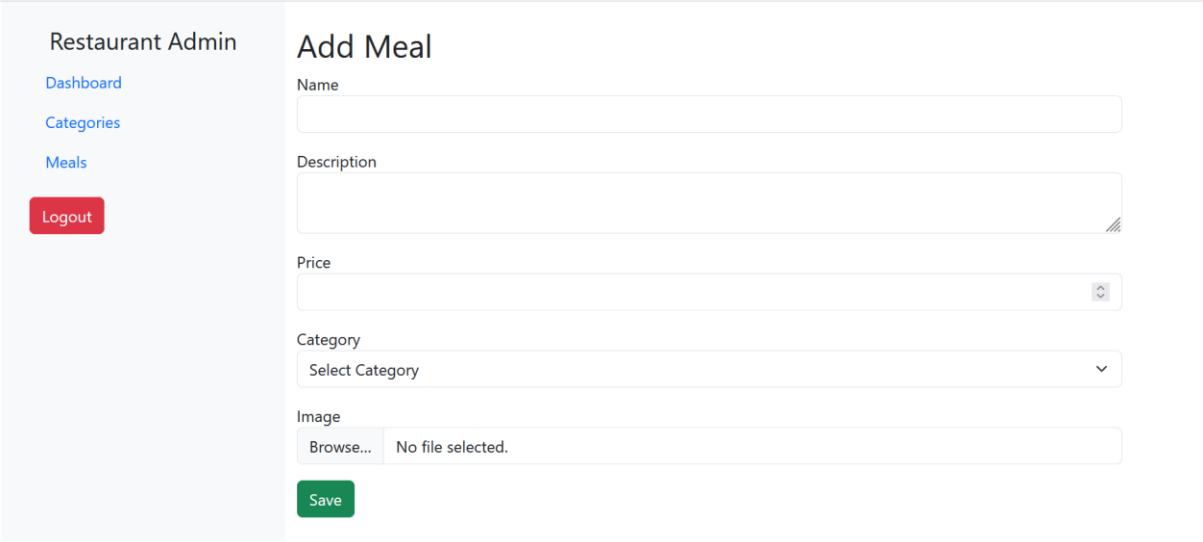
```
</table>
@endsection
```

6.4 Add new category MealController Methods

In MealController Add create methods to display create form

```
public function create() {
    $categories = Category::all();
    return view('meals.create', compact('categories'));
}
```

In resources\views\meals\create.blade.php




```
@extends('layouts.admin-layout')
@section('title', 'Create Meal')
@section('content')
    <h2>Add Meal</h2>

@if ($errors->any())
    <div class="alert alert-danger">
        <ul>
            @foreach ($errors->all() as $error)
                <li>{{ $error }}</li>
            @endforeach
        </ul>
    </div>
@endif
```

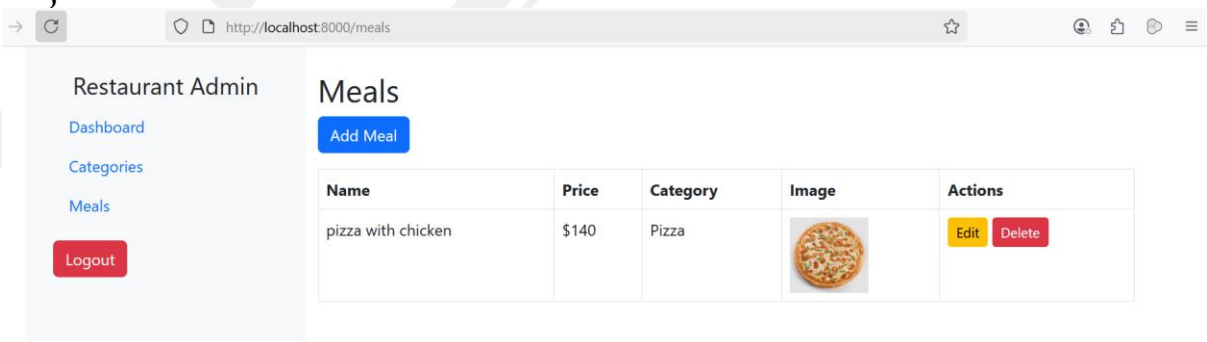
```
<form method="POST" action="{{ route('meals.store') }}"
enctype="multipart/form-data">
  @csrf
  <div class="mb-3">
    <label>Name</label>
    <input type="text" name="name" class="form-control" >
  </div>
  <div class="mb-3">
    <label>Description</label>
    <textarea name="description" class="form-control"></textarea>
  </div>

  <div class="mb-3">
    <label>Price</label>
    <input type="number" name="price" class="form-control" >
  </div>

  <div class="mb-3">
    <label>Category</label>
    <select name="category_id" class="form-select" >
      <option>Select Category</option>
      @foreach($categories as $category)
        <option value="{{$category->id}}">
          {{ $category->name }}
        </option>
      @endforeach
    </select>
  </div>

  <div class="mb-3">
    <label>Image</label>
    <input type="file" name="image" class="form-control">
  </div>
      <button type="submit" class="btn btn-success">Save</button>
  </form>
@endsection
```

In Terminal

```
php artisan storage:link
```

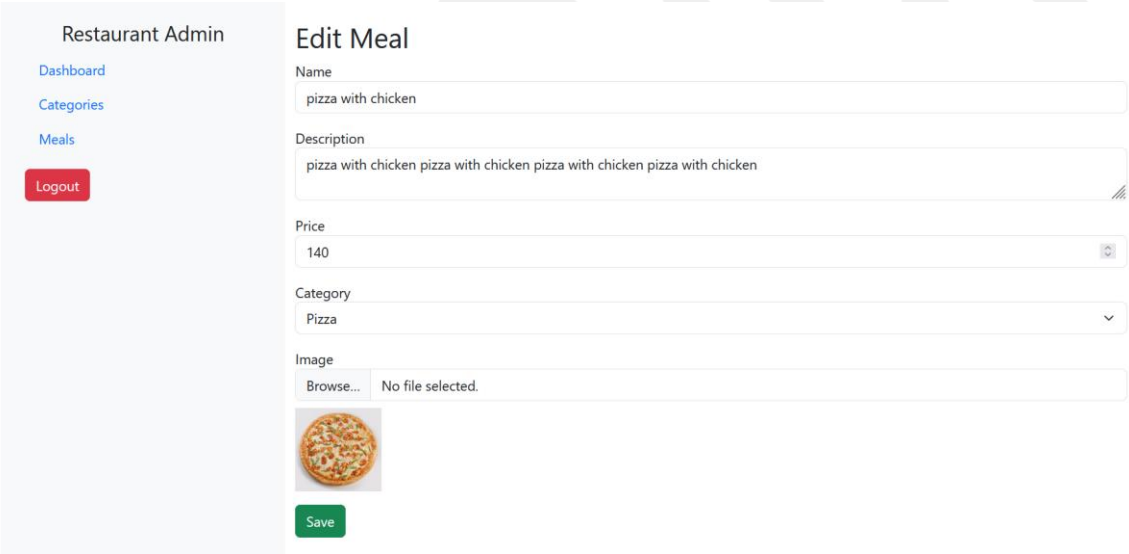
In MealController Add store method to store data

```
use Illuminate\Support\Facades\Storage;
    public function store(Request $request){
     $request->validate([
        'name' => 'required',
        'description' => 'required',
        'price' => 'required|numeric',
        'category_id' => 'required|exists:categories,id',
        'image' => 'image|mimes:jpg,png,jpeg|max:2048',
    ]);

    $image = $request->file('image')?->store('meals', 'public');

    Meal::create([
        'name' => $request->name,
        'description' => $request->description,
        'price' => $request->price,
        'category_id' => $request->category_id,
        'image' => $image,
    ]);

    session()->flash('success', 'Meal created successfully.');
    return redirect()->route('meals.index');
    }
```



In MealController In edit method

```
public function edit(Meal $meal) {
    $categories = Category::all();
    return view('meals.edit', compact('meal', 'categories'));
}
```



In resources\views\meals\edit.blade.php

```
@extends('layouts.admin-layout')
@section('title', 'Edit Meal')
@section('content')
```

```
<h2>Edit Meal</h2>
```

```
@if ($errors->any())
    <div class="alert alert-danger">
        <ul>
            @foreach ($errors->all() as $error)
                <li>{{ $error }}</li>
            @endforeach
        </ul>
    </div>
@endif
```

```
<form method="POST" action="{{route('meals.update', $meal->id)}}') }}" enctype="multipart/form-data">
  @csrf
  @method('PUT')
  <div class="mb-3">
    <label>Name</label>
    <input type="text" name="name" value="{{$meal->name}}" class="form-control" >
  </div>

  <div class="mb-3">
    <label>Description</label>
    <textarea name="description" class="form-control">{{ $meal->description }}</textarea>
  </div>

  <div class="mb-3">
    <label>Price</label>
    <input type="number" name="price" value="{{$meal->price}}" class="form-control" >
  </div>

  <div class="mb-3">
    <label>Category</label>
    <select name="category_id" class="form-select" >
      @foreach ($categories as $category)
        <option value="{{ $category->id }}" {{ $category->id == $meal->category_id ? 'selected' : '' }}>{{ $category->name }}</option>
      @endforeach
    </select>
  </div>

  <div class="mb-3">
    <label>Image</label>
    <input type="file" name="image" class="form-control">
    @if(isset($meal) && $meal->image)
```

```
    <img src="{{ asset('storage/' . $meal->image) }}" width="100"
class="mt-2">
   @endif
  </div>

      <button type="submit" class="btn btn-
success">Update</button>
   </form>
@endsection
```

In CategoryController in update method

```
public function update(Request $request, Meal $meal){
        $request->validate([
       'name' => 'required',
       'description' => 'required',
       'price' => 'required|numeric',
       'category_id' => 'required|exists:categories,id',
       'image' => 'image|mimes:jpg,png,jpeg|max:2048',
   ]);

   if ($request->hasFile('image')) {
       Storage::disk('public')->delete($meal->image);
       $image = $request->file('image')->store('meals', 'public');
       $meal->image = $image;
   }

   $meal->update([
       'name' => $request->name,
       'description' => $request->description,
       'price' => $request->price,
       'category_id' => $request->category_id,
   ]);

   session()->flash('success', 'Meal created successfully.');
   return redirect()->route('meals.index');
   } // update
```

In CategoryController in destroy method

```
    public function destroy(Meal $meal) {
        Storage::disk('public')->delete($meal->image);
        $meal->delete();
        session()->flash('success', 'Meal deleted successfully.');
        return redirect()->route('meals.index');
    } // destroy
```

Section 7: Customer Frontend – View Menu & Filter by Category

By the end of this section, students will be able to:

- Displays meals with images and prices
- Filters meals by selected category

Add Methods to HomeController

```
use App\Models\Meal;
use App\Models\Category;
```

```
class HomeController extends Controller{
    // Show all meals on the homepage
    public function index(){
        $meals = Meal::with('category')->latest()->get();
        $categories = Category::all();
        return view('home', compact('meals', 'categories'));
    }

    // Filter meals by category
    public function category($id){
        $meals = Meal::where('category_id', $id)->get();
        $categories = Category::all();
        return view('home', compact('meals', 'categories'));
    }
}
```

in routes/web.php

Make sure these routes go **outside the auth middleware group**, since they are public:

```
use App\Http\Controllers\HomeController;
```

```
Route::get('/', [HomeController::class, 'index'])->name('home');
Route::get('/category/{id}', [HomeController::class, 'category'])->name('category.filter');
```

In resources\views\home.blade.php

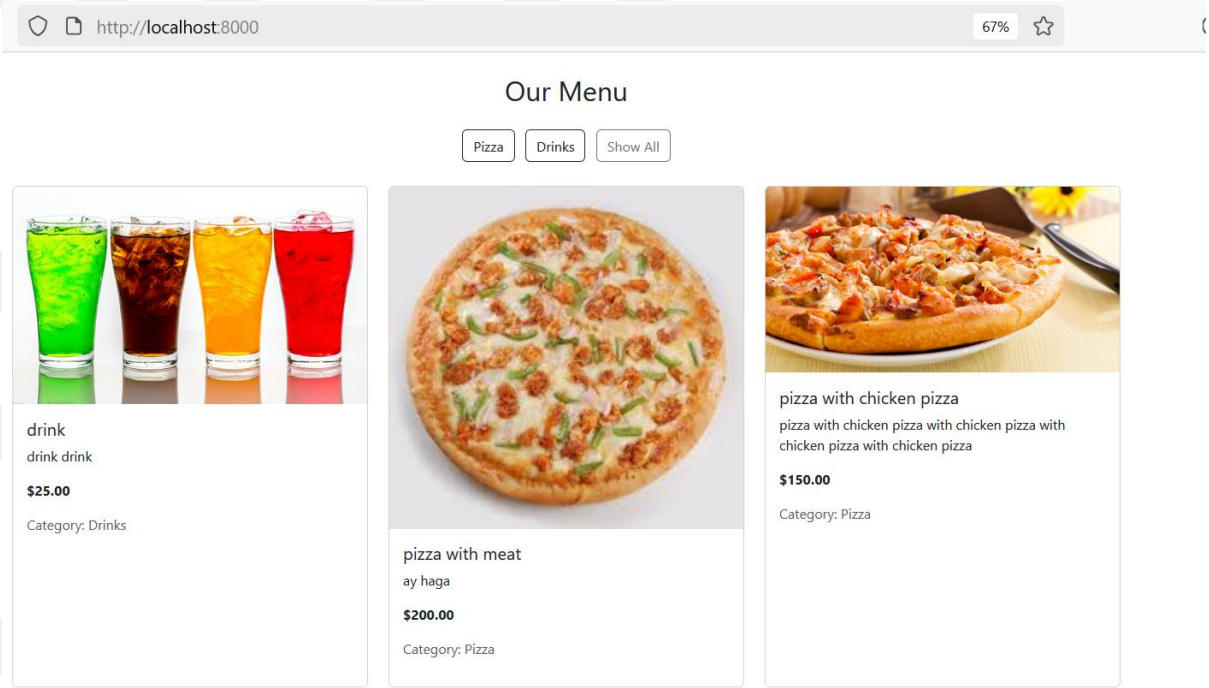
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Restaurant</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>
  <div class="container">
   <h2 class="text-center my-4">Our Menu</h2>
```

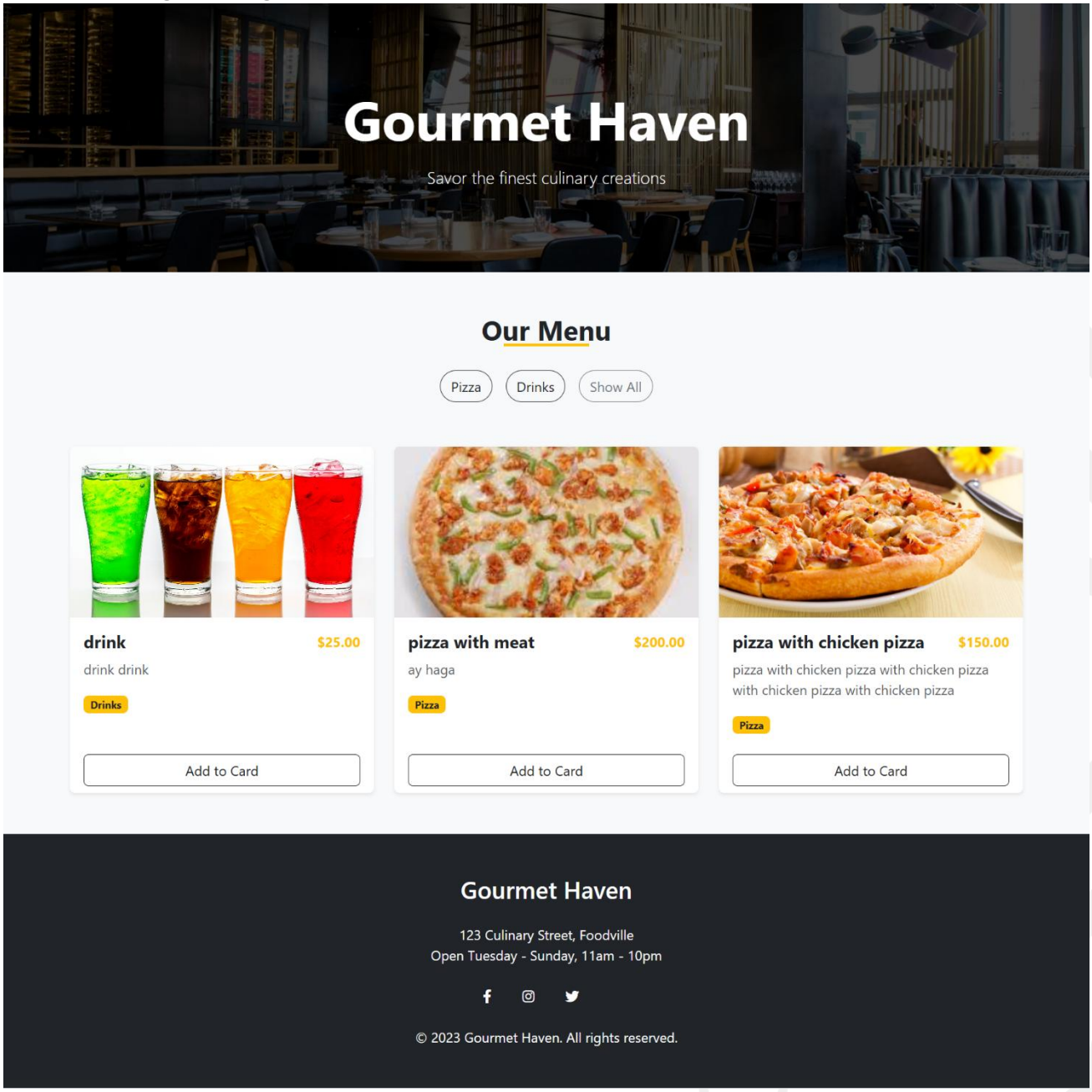
```
<div class="text-center mb-4">
  @foreach($categories as $category)
    <a href="{{ route('category.filter', $category->id) }}" class="btn btn-outline-dark m-1">
      {{ $category->name }}
    </a>
  @endforeach
  <a href="{{ route('home') }}" class="btn btn-outline-secondary m-1">Show All</a>
</div>
```

```
<div class="row">
  @forelse($meals as $meal)
    <div class="col-md-4 mb-4">
```

```
      <div class="card h-100">
        @if($meal->image)
          <img src="{{ asset('storage/' . $meal->image) }}" class="card-img-top" alt="{{ $meal->name }}">
        @endif
```

```
      <div class="card-body">
        <h5 class="card-title">{{ $meal->name }}</h5>
        <p class="card-text">{{ Str::limit($meal->description, 100)
}}</p>
        <p class="fw-bold">${{ number_format($meal->price, 2)
}}</p>
        <p class="text-muted">Category: {{ $meal->category->name
}}</p>
      </div>
    </div><!-- card -->

  </div> <!-- col -->
  @empty
    <p class="text-center">No meals available for this category.</p>
  @endforelse
</div><!-- row -->
  </div> <!-- container -->
</body>
</html>
```



Some challenges for students

- Create show page for meal with logic code
- Change design